

حلول ورقة العمل 06: الأتمتة وحماية الجودة

Solutions · Worksheet 06 — Automation & Quality Gates

Pass mark / معيار النجاح
70 نقطةMandatory evidence / أدلة إلزامية
8 بنودMarking time / زمن التصحيح المقترح
10-8 دقائق لكل طالبTotal / التقييم الكلي
100 نقطة

نسخة المدرّب — نماذج الحلول والتقييم / Instructor copy — do not distribute

ما تقيسه هذه الورقة What this worksheet measures

تقيس هذه الورقة تفكيراً هندسياً لا مهارة استخدام أدوات: هل يفهم الطالب أن الجودة تُفرض تقنياً؟ وهل يدرك أن الفحص نفسه يحتاج اختباراً؟ الطالب الذي يكتشف أن الفحص كان يمرّ على حالة معطوبة (نتيجة خطأ في نمط البحث) قد وصل إلى ما نريده بالضبط — وهذه فرصة تعليمية عالية القيمة، فلا تُفوّتها بالسكوت عنها.

EN

This worksheet measures engineering thinking: quality enforced by tooling, and checks that are themselves tested. The student who discovers that the original marker check silently passed a broken case has reached the target insight — treat it as a highlight, not an error.

جدول التقييم المُجمّع Marking rubric

# المهمة	النقاط معيار القبول	حساسية
1 قراءة سير العمل	12 جدول التحليل مكتمل + فهم دور checkout	متوسطة
2 الفحص المحلي	15 مخرجات ci-local كلها ✓ ومن جهازه	عالية (دليل تنفيذ)
3 اكسر وشاهد الفشل	18 ثلاث حالات معطوبة + دليل فشل حقيقي	الأهم
4 أصلح حتى الأخضر	15 مخرجات ✓ + رابط فحص ناجح على GitHub	عالية
5 حماية الفرع وCODEOWNERS	15 قواعد مفعلة + جدول المسؤوليات + مراجع تلقائي	متوسطة
6 الإصدار v1.0.0	15 وسم مع تعليق + رفع الوسم + CHANGELOG + Release	متوسطة
— الأسئلة (س1-س4)	10 2.5 لكل سؤال	فهم

قاعدة التقييم الخاصة بهذه الورقة Special marking rule

أعلى درجة تذهب لمن كسر المشروع فعلاً. الطالب الذي نفّذ الحالات المعطوبة والتقط دليل فشلها يثبت أنه فهم الأتمتة. أما من «قرأ» الملف ونسخ مخرجاته» فيأخذ ثلث الدرجة فقط في المهمة 3، ولو كانت ورقة سليمة الشكل.

إجابات المهام Task solutions

1 قراءة سير العمل

Reading the workflow

★ 12 نقطة

◆ فهم

الإجابات النموذجية للجدول:

القيمة في مشروعنا

معناه

المفتاح

validate

اسم سير العمل

name

المفتاح	معناها	القيمة في مشروعنا
on:	الأحداث المشغلة	pull_request إلى main · push إلى main · workflow_dispatch (يدوي)
jobs:	المهام	validate مهمة واحدة اسمها
runs-on	نظام الخادم	ubuntu-latest
steps	الخطوات بالترتيب	5 خطوات: checkout · setup-node · build-index --check · validate · members · فحص علامات التعارض
actions/checkout	يُنزّل نسخة المستودع على الخادم	—
actions/setup-node	يُثبّت إصدار Node المطلوب (20)	—

معيّار الدرجة: 8 نقاط للجدول الصحيح (خطأ واحد مسموح)، 4 نقاط لشرح دور `actions/checkout` (يُنزّل الكود، بدون لا يوجد ما يُفحص).

أخطاء متوقعة: كتابة عدد الخطوات 3 (خلط بين عدد الفحوص وعدد الخطوات) ← وأشر إلى الفرق؛ إجابة أن `runs-on` هو «جهاز الطالب» ← تصحيح مباشر لأهمية عزل بيئة الفحص.

2 الفحص المحلي

Local CI run

★ 15 نقطة

◆ تنفيذ

المخرجات المتوقعة (من حراز الطالب، ثلاث علامات ✓):

≡ فحص محلی ریشه GitHub Actions / Local CI simulation ≡

```

نجح ✓ ... index up to date / الفهرس مُحدّث
[build-index] ✓ الفهرس محدّث بالفعل.

نجح ✓ ... member cards valid / بطاقات الأعضاء صحيحة
---بطاقة، 0 خطأ، 0 تحذير 3 Result: / النتيجة ---

نجح ✓ ... no conflict markers / لا علامات تعارض

All checks passed. / (git push) كل الفحوص نجحت - آمن للرفع ✓

```

أخطاء متوقعة وخصوصياتها:

الحالة	الدلالة	الخصم
نسخ مخرجات الورقة (عدد البطاقات يختلف عن واقعه)	دليل غير شخصي	-10 مع إعادة
عنده فشل حقيقي ولم يصلحه	لم يُكمل المهمة	6 من 15
شغل <code>ci-local</code> ثم تجاهل الفشل ورفع	يفهم الأداة ولا يحترمها	5 من 15 + مناقشة
أضاف الأمر إلى <code>git push &&</code> (مبادرة)	ممارسة مهنية ممتازة	+2 تقدير

Quick quality signal مؤشّر جودة سريع فى الجولة

اسأل الطالب: «ماذا يعني `exit=0` في نهاية المخرجات؟» الجواب المطلوب: أن العملية نجحت، وأن نظام CI يقرأ هذا الرمز (لا النص) ليحدد حالة الفحص. من فهم «رمز الخروج» يفهم لماذا تفشل خطوط الأنابيب (pipelines) في الفرق الحقيقية.

★ 18 نقطة

◆ إتقان

الحالات الثلاث ومخرجاتها الحقيقية:

(أ) فهرس قديم (نسيان `build-index`):

```

❌ فشل ... index up to date / الفهرس مُحدّث
[build-index] ❌ شغل
❌ فشل ... member cards valid / بطاقات الأعضاء صحيحة
❌ data/members-index.json: البطاقة "ali-h.json" شغل
node tools/build-index.mjs: غير مُفهرسة - شغل
---بطاقة، 1 خطأ، 0 تحذير 4 Result: / النتيجة ---

. من 3 فحص فشل - لا ترفع قبل الإصلاح 2 ❌

```

(ب) بطاقة بمحتوى مخالف:

```

❌ omar-qa.json: اسم مستخدم غير صالح / invalid username: "Omar_QA"
⚠️ omar-qa.json: دور غير معتاد / unusual role: "سوبرمان"
❌ omar-qa.json: الامر المفضل يجب ان يبدأ بـ "git"
❌ omar-qa.json: tags must be an array / يجب أن تكون مصفوفة tags
❌ sara-design.json: طول الرسالة 5 - يجب أن يكون بين 10 و140 حرفاً
---بطاقة، 5 خطأ، 2 تحذير 4 Result: / النتيجة ---

```

(ج) علامات تعارض باقية:

```

❌ فشل ... no conflict markers / لا علامات تعارض
./sandbox/team-slogan.txt:2:<<<<<< HEAD
./sandbox/team-slogan.txt:4:=====
./sandbox/team-slogan.txt:6:>>>>>> feature/omar-slogan

. من 3 فحص فشل - لا ترفع قبل الإصلاح 1 ❌

```

معيّار الدرجة (18 نقطة): 4 نقاط لكل حالة مكتشفة بمخرجها الحقيقي (12)، و6 نقاط لدليل الحالة الفاشلة على GitHub (رابط Actions أو ذكر حالة `1 failing check` ❌)، بشرط أن يذكر جملة `Merging is blocked`.

أخطاء متوقعة:

الخطأ	الدلالة	الدرجة
جَرَّب حالة واحدة فقط	تجربة ناقصة	6 من 12
اكتفى بقراءة الملف ولم يجَرَّب	لم يفهم قيمة الأتمة	3 من 18
لم ينتبه لأن الحالة (ج) تحتاج نطاق بحث صحيحاً هذا مقصود! انظر أدناه لا خصم		
كسر المستودع ولم يُصلحه في المهمة 4	لم يُكمل الدورة	8 من 18

الدرس المصمّم بعناية: فحص ضعيف يخدع صاحبه *The designed trap: a weak check lies*

الإصدار الأول من فحص علامات التعارض كان يستخدم نمطاً غير دقيق:

هذا المشروع):
`grep -rInE "^(<<<<<<|>>>>>>|=====)$"` مع حصر البحث بامتدادات محددة. الثغرات الثلاث (مُختبرة فعلياً على

1. النطاق: `sandbox/team-slogan.txt` — ملف مختبر التعارض في الورقة 05 — خارج نطاق البحث، فكان الفحص أخضر على مستودع مكسور. هذه هي الثغرة الكبرى.

2. شكل العلامة: النمط يشترط أن تكون العلامة وحدها في السطر، والعلامة الحقيقية `<<<<<< HEAD` تحمل اسم الفرع، وكذلك `feature/x >>>>>>`؛ فالنمط لا يلتقط إلا السطر الأوسط `=====` وحده — وأي ملف فُقد فيه السطر الأوسط أو أُزيح يمرّ.

3. إيجابية كاذبة: سطر `=====` فاصل في ملف Markdown سليم يُظهر «علامات تعارض» فيتعوّد الفريق على تجاهل الإنذار (Alarm fatigue) — وهذا ما جعل الثغرة الأولى تعيش طويلاً.

النمط الصحيح بعد الإصلاح:

```
grep -rInE "^(<{7}={7}>{7}) (|$)" --exclude-dir=.git --exclude-dir=node_modules .
```

كيف تستخدم هذا في التقييم: - الطالب الذي اكتشف أن الفحص كان يمرّ خطأً: **5+ تقدير** واذكره في الفريق، فهذا تفكير نقدي حقيقي (Critical thinking) نادر في هذا العمر التدريبي. - الطالب الذي مرّ على الحالة (ج) ولم يلاحظها: لا خصم، لكن اترح عليه السؤال ليقودك بنفسه إلى الاستنتاج: «هل جربت أن تضع علامة `<<<<<< HEAD` داخل ملف بامتداد `.txt`؟». - القاعدة التي تُعلن للصّف: كل فحص جديد يجب أن يُختبر على حالة معطوبة قبل اعتماده.

4 أصلح حتى يصبح أخضر

Fix until green

★ 15 نقطة

◆ إتقان

الإصلاح الصحيح لكل حالة:

الحالة	الإصلاح	الأمر
فهرس قديم	أعد التوليد وأضف الملفين الناتجين	<code>node tools/build-index.mjs</code> ثم <code>git add data/members-index.json data/members-bundle.js</code>
بطاقة مخالفة	صحّح الحقل المذكور في رسالة الفحص	تحرير الملف ثم إعادة <code>ci-local</code>
علامات تعارض	احذف <code><<<<<<</code> و <code>=====</code> و <code>>>>>>></code> واترك النتيجة النهائية	<code>git add</code> ثم الالتزام

المخرجات المتوقعة بعد الإصلاح:

```
$ node tools/build-index.mjs
[build-index] ✓ بطاقة 4 فهرس: تم توليد الفهرس.
```

```
$ node tools/ci-local.mjs
الفرس مُحدّث ... ✓ نجح
بطاقات الأعضاء صحيحة ... ✓ نجح
لا علامات تعارض ... ✓ نجح
✓ (git push) / All checks passed. (exit=0)
كل الفحوص نجحت - آمن للرفع
```

وعلى GitHub بعد الدفع:

```
✓ All checks have passed
1 successful check
validate / فحص البطاقات والفهرس - Successful in 12s
● This branch has no conflicts with the base branch.
```

معيّار الدرجة: 6 نقاط لمخرجات ✓ من جهازه، 5 نقاط لرباط/لقطة الفحص الأخضر على 4، GitHub نقاط لمقارنة زمن الفحص (الخادم أبداً عادة - الجواب المتوقع: على الخادم 10-30 ثانية لأنه ينزل المستودع ويثبت Node من الصفر).

أخطاء متوقعة: من حلّ بالحذف بدل التصحيح (حذف البطاقة المخالفة ليعبر الفحص) ← **6- وتنبيه صريح:** «الحذف لإخفاء الخطأ يعطل الغرض من الفحص»، وهي مخالفة تتكرر في المشاريع الحقيقية وتُعدّ سلوكاً غير مهني.

5 حماية الفرع وملّك الكود

Protection and code owners

★ 15 نقطة

◆ فهم تنظيمي

الإجابات النموذجية للجدول:

الملف / المسار	المراجع الإلزامي	السبب
css/ و js/	قائد الفريق	تعديلهما يؤثر على عرض كل البطاقات
data/members/<username>.json	الطالب نفسه (مالكه)	لا يحتاج مراجعة إلزامية من القائد لأنه لا يؤثر على غيره
data/site-config.json	قائد الفريق	إعدادات الموقع العامة (اسم الفريق، الوصف)
tests/ و tools/	قائد الفريق	أدوات الفحص تحمي جودة الجميع - تعديلها قد يعطل الحماية
القواعد التي يجب أن تكون مفعلة (تحقق نظراً في إعدادات الطالب أو مستودع الصف): - Require a pull request before merging + Require status checks to pass before merging + Require approvals: 1 - Require conversation resolution - Do not allow bypassing - (validate / فحص البطاقات والفهرس / pushes)		

دليل «المراجع التلقائي»: في طلب سحب يعدّل js/app.js يجب أن يظهر تحت Reviewers اسم المالك مع لافتة requested automatically by CODEOWNERS.

معيّار الدرجة: 8 نقاط للجدول (2 لكل سطر)، 4 نقاط لقائمة القواعد المفعلة، 3 نقاط لدليل المراجع التلقائي.

أخطاء متوقعة: - الطالب لا يملك صلاحية تفعيل الحماية (مشروع صقيّ بصلاحيات محدودة) ← اعتمد لقطة إعدادات مستودع الصف التي تعرضها أنت على الشاشة، ومنحه الدرجة كاملة على التحليل. - من كتب «كل الملفات يراجعها القائد» ← يفقده التمييز بين الملف الفردي والمشارك (-2). - من جعل CODEOWNERS يُلزم الجميع حتى على ملفات البطاقات ← يبطئ الفريق بلا داعٍ؛ اشرح المبدأ: المراجعة الإلزامية تُوقّر للمخاطر، لا للشكليات.

المخرجات النموذجية:

```
$ git tag -a v1.0.0 -m "الإصدار الأول: موقع دليل فريق الصف مع 4 بطاقات وسير تحقق تلقائي"
$ git tag -l
v1.0.0

$ git describe --tags
v1.0.0

$ git show v1.0.0 --stat --no-patch
tag v1.0.0
Tagger: Workshop Instructor <instructor@example.com>
Date: Tue Oct 6 20:57:47 2026 +0000

الإصدار الأول: موقع دليل فريق الصف مع 4 بطاقات وسير تحقق تلقائي

commit e20b6644b539bbb22200dde76ea5e5015d826b86
Merge: ccbdca2 eb1df06
Merge pull request #32 from ali-h/feature/ali-card

$ git tag -n1
v1.0.0      الإصدار الأول: موقع دليل فريق الصف مع 4 بطاقات وسير تحقق تلقائي
v1.1.0      ميزة الفلتر بالوسوم

$ git push origin v1.0.0
* [new tag]          v1.0.0 -> v1.0.0

$ git ls-remote --tags origin
1105c78548ddcafca65f03d9ea10f58ecddd0d15      refs/tags/v1.0.0
e20b6644b539bbb22200dde76ea5e5015d826b86      refs/tags/v1.0.0^{}
```

معيار الدرجة (15): 5 نقاط لوسم مع تعليق (-a) و (-m) - والوسم الخفيف (git tag v1.0.0) يأخذ 2 فقط لأن الإصدارات الرسمية تحتاج وصفاً ومصدراً؛ 4 نقاط لرفع الوسم (git push origin v1.0.0) أو (--tags)؛ 3 نقاط لإنشاء Release على GitHub برابط صالح؛ 3 نقاط لتحديث CHANGELOG.md .

أخطاء متوقعة:

الخطأ	الدلالة	الدرجة
git push فقط وسمي «رفعت الإصدار» لا يعرف أن الوسوم لا تُرفع مع الفروع 8 من 15	وسم خفيف بلا تعليق	10 من 15
لم يحدّث CHANGELOG.md	ناقص المتانة	12 من 15
أطلق v2.0.0 بلا تغيير يكرس التوافق	نسي توثيق الإصدار	9 من 15 + شرح
سجل إصداراً بلا دليل (رابط)	خلل في فهم SemVer	5 من 15
	لا يمكن التحقق	

تصحيح ذكي: فحص `ls-remote` Smart verification

لا تصدّق لقطة الشاشة وحدها. أمر واحد يتحقق من وجود الوسم على الخادم فعلاً:

```
git ls-remote --tags origin
```

إن لم يظهر `refs/tags/v1.0.0` فالوسم محلي فقط ولم يُرفع — وهذه أسرع طريقة لكشف الخطأ.

إجابات الأسئلة Knowledge-check answers

س1 الفرق بين الحماية و CI

Protection vs. CI

★ 2.5 نقطة

◆ فهم

الجواب: فحص CI يكتشف الخلل ويُعلن عنه (يمنع الدمج من حيث النتيجة إذا كان مطلوباً في الحماية)، أما حماية الفرع فهي **قواعد الوصول** نفسها: تمنع الدفع المباشر إلى `main`، وتُلزم بطلب سحب وموافقة مراجع ومحددات محلولة، وتمنع القوة وإعادة كتابة التاريخ. أي أن CI هو «حكم الجودة»، والحماية هي «نظام الدخول والصلاحيات» — ولا تكتمل الفائدة بأحدهما وحده: بلا حماية يمكن تجاوز الفحص بالدفع المباشر، وبلا فحص تُدمج أخطاء ميكانيكية.

س2 اخلل المنطق في «الفحص يمرّ دائماً»

The flawed assumption

★ 2.5 نقطة

◆ تحليل

الجواب: العبارة تفترض أن الفحص قادر على الكشف، وهذا غير مُثبت — فالفحص الذي يمرّ دائماً قد يكون **ضعيفاً** (نمط بحث خاطئ، نطاق مستثنى، شرط لا يتحقق أبداً)، لا أن المشروع سليم. التشبيه: إطفاء إنذار الحريق لا يعني عدم وجود حريق. وطريقة اختبار: أن تُنشئ حالة معطوبة عن قصد (اختبار سلبي) وتتأكد أن الفحص يفشل فيها بـ `exit=1`، وتكرر ذلك عند أي تعديل على الفحص. **وهذا بالضبط ما حدث في هذا المشروع:** فحص علامات التعارض كان يمرّ على ملف فيه علامات حقيقية.

س3 وسم بسيط مقابل وسم مع تعليق

Lightweight vs annotated tags

★ 2.5 نقطة

◆ فهم

الجواب: الوسم البسيط مجرد مؤشر (pointer) إلى التزام بلا بيانات إضافية. الوسم مع التعليق كائن مستقل في Git يحتوي: من أنشأه، تاريخ الإنشاء، ورسالة كاملة. لذلك يُستحسن **الوسم مع التعليق** (`-a`) للإصدارات، لأنه يُظهر في `git describe --tags` ويحمل سياق الإصدار ويمكن توقيعه رقمياً (GPG)، وتُبنى عليه Releases على GitHub بصورة أنظف.

س4 فائدة منع الدفع القسري

Why block force pushes

★ 2.5 نقطة

◆ تحليل

الجواب: لأن `--force` يعيد كتابة تاريخ `main` ويستبدله بتاريخ دفعته، فيضيع عمل أي زميل دُمج بعد النسخة التي استند إليها، وتنكسر روابط الالتزامات في طلبات السحب والمراجعات والإصدارات. منعه على `main` يضمن أن التاريخ المشترك لا يُعاد كتابته أبداً،

وأن إصلاح أي خطأ يمر بالطريق الوحيد الآمن: `git revert` أو التزام تصحيحي عبر طلب سحب.

ملخص التصحيح *Marking summary*

سجل الدرجات *Grade sheet*

الاسم م 1/ 12 م 2/ 15 م 3/ 18 م 4/ 15 م 5/ 15 م 6/ 15 أسئلة 10/ المجموع 100/ رابط CI تقدير المبادرة

قياس أثر هذه الورقة على الفريق *Measuring the unit's impact*

مؤشر عملي بسيط يقيس أثر الأتمتة على صفك كله: 1. قبل الورقة: احسب عدد طلبات السحب التي فشلت أو احتاجت تصحيحاً من المراجع (خطأ JSON أو فهرس قديم). سجّلها. 2. بعد الورقة بأسبوع: احسب نفس الرقم. 3. الانخفاض المتوقع 60–80% في الأخطاء الميكانيكية. اعرض النتيجة على الطلاب – هذا أبلغ دليل على قيمة الأتمتة، ويجعلهم يطلبون إضافة فحوص جديدة بأنفسهم (وهذا هو الهدف النهائي: ثقافة الجودة لا مهمة ورقية).

توسعات لمن أنهى بسرعة *Fast-finisher extensions*

- أضف فحصاً جديداً إلى `validate-members.mjs` يتأكد أن `tags` لا تحتوي أكثر من 4 وسوم، ثم اختبره بحالة معطوبة قبل اعتماده.
- أضف إلى `validate.yml` خطوة تنشر الموقع على GitHub Pages آلياً عند كل وسم إصدار (`job deploy`) مع `(if: startsWith(github.ref, 'refs/tags/'))`.
- أضف فحص `CHANGELOG` يتأكد أن آخر إصدار في `CHANGELOG.md` يطابق آخر وسم في Git – تمرين ممتاز على دمج الأتمتة مع التوثيق.